CS 101 - Algorithms & Programming I

Spring 2026 - Lab 3

Due: Week of February 23, 2026

*Remember the **honor code** for your programming assignments.*
*For all labs, your solutions must conform to the CS101 style **guidelines**!*
All data and results should be stored in variables (or constants) with meaningful names.

The objective of this lab is to learn how to program simple and complex decisions by using if/else statements. Remember that analyzing your problems and designing them on a piece of paper *before* starting implementation/coding is always a best practice.

In this particular lab, do **not** use a switch-case statement instead of an if-else statement!

0. Setup Workspace

Start VSC and open the previously created workspace named labs_ws. Now, under the labs folder, create a new folder named lab3.

In this lab, you are to have three Java classes/files (under labs/lab3 folder) as described below. A fourth Java file containing the revision should go under this folder as well. We expect you to **submit a total of 4 individual files,** including the revision, **without compressing** them. Do *not* upload other/previous lab solutions in your submission. The user inputs in the sample runs are shown in blue. In this particular lab, **do not use a switch-case statement instead of an if-else statement!**

0. Setup Workspace

Start VSC and open the previously created workspace named `labs_ws`. Now, under the `labs` folder, create a new folder named `lab3`. In this lab, you are to have three Java classes/files (under `labs/lab3` folder) as described below. A fourth Java file containing the revision should go under this folder as well. We expect you to submit a total of **4 individual files**, including the revision, without compressing them. Do not upload other/previous lab solutions in your submission.

The user inputs in the sample runs are shown in **blue**.

1. Movie Ticket Pricing System

Create a Java program named **`Lab03_Q1.java`** in the `lab3` folder. This program will calculate the price of a movie ticket based on customer age, movie type, and viewing time. The program should determine the final price and provide a purchase summary.
The program will ask the user to enter:

- Customer age (integer)
- Movie type (String): "regular", "3D", or "IMAX"
- Show time (String): "matinee" (before 5 PM) or "evening" (5 PM or later)
- Premium membership (boolean): true or false

Pricing Logic:
The base ticket prices are:

- Regular movie: $10.00
- 3D movie: $15.00
- IMAX movie: $18.00

Age-based discounts (applied to base price):

Age Category	Discount
Children (age < 12)	40% discount
Students (age 12-25)	25% discount
Adults (age 26-64)	No discount
Seniors (age ≥ 65)	35% discount

Additional modifiers:

- Matinee shows: Additional 20% discount on final price
- Premium membership: Additional 10% discount on final price
- Evening IMAX shows: Add $3.00 surcharge (applied after discounts)

Important rules:

- Children under 5 cannot watch IMAX movies (display appropriate error message)
- Maximum discount cannot exceed 60% of the original base price

Output:

The program should display:

- The base price
- The final price after all discounts/surcharges
- Total savings (if any)
- A message about age category and applied discounts

Sample Runs:

Sample Run 1 (Student with matinee and premium):

```
Enter customer age: 22
Enter movie type (regular/3D/IMAX): 3D
Enter show time (matinee/evening): matinee
Premium member? (true/false): true

Base Price: $15.00
Student Discount Applied: 25%
Matinee Discount Applied: 20%
Premium Member Discount Applied: 10%
Final Price: $8.10
Total Savings: $6.90
Enjoy your movie!
```

Sample Run 2 (Senior with evening IMAX):

```
Enter customer age: 70
Enter movie type (regular/3D/IMAX): IMAX
Enter show time (matinee/evening): evening
Premium member? (true/false): false

Base Price: $18.00
Senior Discount Applied: 35%
Evening IMAX Surcharge Applied: $3.00
Final Price: $14.70
Total Savings: $3.30
Enjoy your movie!
```

Sample Run 3 (Child under 5 trying IMAX - Error case):

```
Enter customer age: 4
Enter movie type (regular/3D/IMAX): IMAX
Enter show time (matinee/evening): matinee
Premium member? (true/false): false

Error: Children under 5 cannot watch IMAX movies.
```

2. Travel Package Eligibility System

Create a Java program named **`Lab03_Q2.java`** in the `lab3` folder. This program evaluates a customer's eligibility for various travel package deals based on multiple criteria and calculates a personalized offer score.

The program will ask the user to enter:

- Customer age (integer)
- Annual travel budget (double, in dollars)
- Number of previous trips with company (integer)
- Preferred destination type (String): "beach", "mountain", "city", or "adventure"
- Travel season (String): "summer", "winter", "spring", or "fall"
- Number of travelers in group (integer)

Eligibility Score Calculation:

The total eligibility score is calculated by starting with a base score of 50 points and then adding or subtracting points based on the following criteria:

Age Points:

Age Range	Points Added
18-30	+15 points
31-45	+25 points
46-60	+20 points
61-75	+10 points
Others	0 points

Budget Points (maximum 150 points):

- Add 10 points per $1,000 of budget (capped at 150 points)

Loyalty Points (based on previous trips):

Previous Trips	Points Added
0-2 trips	0 points
3-5 trips	+30 points
6-10 trips	+60 points
11+ trips	+100 points

Seasonal Bonus Points:

- Summer beach or adventure: +20 points
- Winter mountain: +20 points
- Spring city: +15 points
- Fall mountain or city: +15 points
- All other combinations: 0 points

Group Size Penalty:

- Deduct 5 points per person in group (larger groups increase complexity)

Destination Preference Bonus:

- Adventure destinations: +25 points if customer age is 18-45
- Beach destinations: +15 points if travel season is summer
- Mountain destinations: +15 points if travel season is winter

Package Eligibility Requirements (ALL conditions must be met):

- Age ≥ 18
- Budget ≥ $2,000
- Number of travelers ≤ 8
- Total eligibility score ≥ 150

Output:

The program should:

- Display the total eligibility score
- If eligible: Display approved message with package tier:
  - Score 150-199: "Silver Package"
  - Score 200-249: "Gold Package"
  - Score 250+: "Platinum Package"
- If not eligible: Display rejection message with ALL specific reasons that apply:
  - Age is below 18
  - Annual budget is below $2,000
  - Group size exceeds 8 travelers
  - Total eligibility score is below 150 points

Sample Runs:

Sample Run 1 (Approved - Silver Package):

```
Enter customer age: 28
Enter annual travel budget: 5000
Enter number of previous trips with company: 7
Enter preferred destination type (beach/mountain/city/adventure): beach
Enter travel season (summer/winter/spring/fall): summer
Enter number of travelers in group: 3

Total eligibility score: 195

The applicant is approved for the travel package.
Package tier: Silver Package
```

Sample Run 2 (Approved - Platinum Package):

```
Enter customer age: 35
Enter annual travel budget: 15000
Enter number of previous trips with company: 12
Enter preferred destination type (beach/mountain/city/adventure): adventure
Enter travel season (summer/winter/spring/fall): summer
Enter number of travelers in group: 2

Total eligibility score: 360

The applicant is approved for the travel package.
Package tier: Platinum Package
```

Sample Run 3 (Not Approved - Multiple Reasons):

```
Enter customer age: 16
Enter annual travel budget: 1500
Enter number of previous trips with company: 1
Enter preferred destination type (beach/mountain/city/adventure): city
Enter travel season (summer/winter/spring/fall): winter
Enter number of travelers in group: 10

Total eligibility score: 15

The applicant is not approved for the travel package. Reason:
Age is below 18.
Annual budget is below $2,000.
Group size exceeds 8 travelers.
Total eligibility score is below 150 points.
```

3. Online Course Management System

Create a Java program named **`Lab03_Q3.java`** in the `lab3` folder. This program simulates a simple online course management system where instructors can log in and manage their courses and students.

Initial System Setup:

The system should be initialized with the following values:

- Account username: "instructor"
- Account password: "teach2024"
- Initial enrolled students: "AliceJohnson, BobWilliams, CarolDavis, "
- Initial courses: "COURSE101:IntroToPython COURSE102:WebDevelopment "

Login Process:

- First, prompt for username
- If username is incorrect: display "Username not found! Goodbye!" and exit
- If username is correct, then prompt for password
- If password is incorrect: display "Incorrect password! Goodbye!" and exit
- If both are correct: show the operations menu

Operations Menu (displayed after successful login):

```
1- Add student
2- Remove student
3- Add course
4- Remove course
5- Logout
```

Feature Details:

Operation 1 - Add Student:

- Prompt the user to enter the student name
- Check if the student already exists in the enrolled students list (case-sensitive comparison)
- If the student exists: display "This student is already enrolled!"
- If the student does not exist: add the student to the list with format "StudentName, "
- Display the updated student list with message "Your students: (student list)"

Operation 2 - Remove Student:

- Prompt the user to enter the name of the student to remove
- Check if the student exists in the enrolled students list
- If the student exists: remove the student from the list
- If the student does not exist: display "No student found with this name!"
- Display the updated student list with message "Your students: (student list)"

Operation 3 - Add Course:

- Prompt the user to enter the course name (can contain spaces, use nextLine())
- Generate a random course code: a random integer between 100 and 999
- Check if the generated code already exists in the courses list
- If the code exists: display "A course with code [code] already exists, cannot add!" and do not add the course
- If the code does not exist: add the course with format "COURSE[code]:[CourseName] "
- Display the updated courses list with message "Your courses: (courses list)"

Operation 4 - Remove Course:

- Prompt the user to enter the course code (e.g., "101", "102")
- Check if a course with that code exists in the courses list
- If the course exists: remove the entire course entry (both code and name)
- If the course does not exist: display "No course found with this code!"
- Display the updated courses list with message "Your courses: (courses list)"

Operation 5 - Logout:

- Display "Logged out successfully!" and terminate the program

Error Handling:

- If the user selects an invalid operation number: display "There is no such operation! Goodbye!" and terminate the program

Sample Runs:

Sample Run 1 (Add Student):

```
Enter your username: instructor
Enter your password: teach2024
1- Add student
2- Remove student
3- Add course
4- Remove course
5- Logout
Select an operation: 1
-- Add Student --
Enter student name: DavidMiller
New student DavidMiller is added!
Your students: (AliceJohnson, BobWilliams, CarolDavis, DavidMiller, )
```

Sample Run 2 (Remove Student):

```
Enter your username: instructor
Enter your password: teach2024
1- Add student
2- Remove student
3- Add course
4- Remove course
5- Logout
Select an operation: 2
-- Remove Student --
Enter student name which you want to delete: BobWilliams
BobWilliams is deleted successfully from students!
Your students: (AliceJohnson, CarolDavis, )
```

Sample Run 3 (Add Course):

```
Enter your username: instructor
Enter your password: teach2024
1- Add student
2- Remove student
3- Add course
4- Remove course
5- Logout
Select an operation: 3
-- Add Course --
Enter course name: Database Systems
New course with code XXX is added!
Your courses: COURSE101:IntroToPython COURSE102:WebDevelopment
COURSEXXX:Database Systems
```

Sample Run 4 (Remove Course):

```
Enter your username: instructor
Enter your password: teach2024
1- Add student
2- Remove student
3- Add course
4- Remove course
5- Logout
Select an operation: 4
-- Remove Course --
Enter course code which you want to delete: 101
The course with code 101 is deleted successfully!
Your courses: COURSE102:WebDevelopment
```

Sample Run 5 (Logout Course):

```
Enter your username: instructor
Enter your password: teach2024
1- Add student
2- Remove student
3- Add course
4- Remove course
5- Logout
Select an operation: 5
Logged out successfully!
```